

RAG (Retrieval-Augmented Generation)

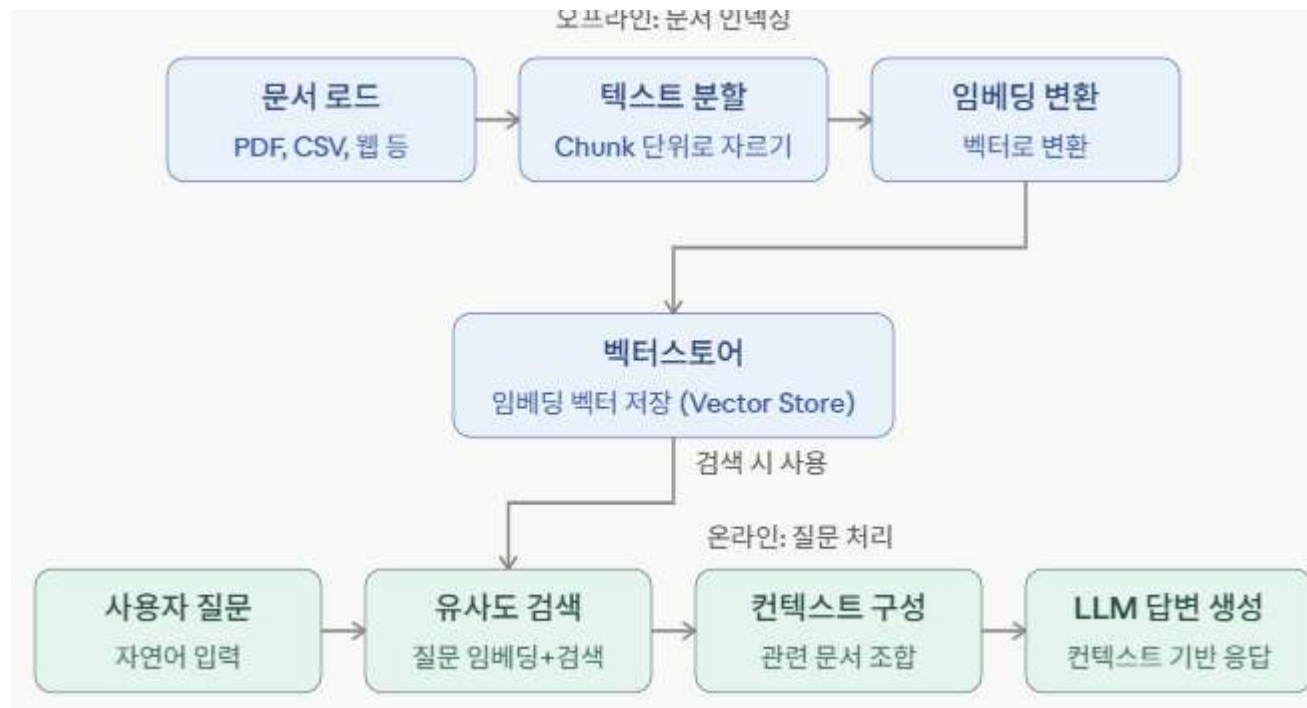
RAG (Retrieval-Augmented Generation)

■ RAG (Retrieval-Augmented Generation)

- 기존의 대규모 언어 모델에 검색 기능을 추가하여, 주어진 질문이나 문제에 대해 더 정확하고 풍부한 정보를 기반으로 답변을 생성
- 모델이 학습 데이터에 포함되지 않은 외부 데이터를 **실시간으로 검색(retrieval)**하고, 이를 바탕으로 답변을 생성 (generation)하는 과정을 포함
- **환각**(생성된 내용이 사실이 아닌 것으로 오인되는 현상)을 **방지**하고, 모델이 최신 정보를 반영하거나 더 넓은 지식을 활용할 수 있게 합니다.
- RAG(Retrieval-Augmented Generation) 파이프라인
 - 데이터 로드 : 외부 데이터 소스에서 정보를 수집하고, 필요한 형식으로 변환하여 시스템에 로드합니다.
 - 텍스트 분할 : 불러온 데이터를 작은 크기의 단위(chunk)로 분할하는 과정
 - 인덱싱 : 분할된 텍스트를 검색 가능한 형태로 만드는 단계
 - 검색 : 사용자의 질문이나 주어진 컨텍스트에 가장 관련된 정보를 찾아내는 과정
 - 생성 : 검색된 정보를 바탕으로 사용자의 질문에 답변을 생성하는 단계

RAG (Retrieval-Augmented Generation)

■ RAG (Retrieval-Augmented Generation)



RAG (Retrieval-Augmented Generation)

■ VectorDB t

Vector Store	패키지 / 클래스
InMemory	langchain_core.vectorstores. InMemoryVectorStore
Local Vector DB	angchain_community.vectorstores. FAISS
Local Vector DB	langchain_chroma. Chroma
Cloud Vector DB	Pinecone, Qdrant , Milvus , Weaviate AstraDBVectorStore , MongoDBAtlasVectorSearch , ElasticVectorSearch , OpenSearchVectorSearch , Redis , AzureSearch , PGVector Clickhouse Neo4jVector Cassandra SingleStoreDB SupabaseVectorStore

RAG (Retrieval-Augmented Generation)

■ Load Data

- `langchain_community.document_loaders` 모듈의 `WebBaseLoader` 클래스를 사용하여 특정 웹페이지(위키피디아 정책과 지침)의 데이터를 가져올 수 있습니다
- 웹페이지의 텍스트 데이터를 추출하여 Document 객체의 리스트로 변환합니다.

```
# Data Loader - 웹페이지 데이터 가져오기
from langchain_community.document_loaders import WebBaseLoader

# 위키피디아 정책과 지침
url =
'https://ko.wikipedia.org/wiki/%EC%9C%84%ED%82%A4%EB%B0%B1%EA%B3%BC:%EC%A0%95%EC%B1%85%EA%B3%BC_%EC%A7%80%EC%B9%A8'
loader = WebBaseLoader(url)

# 웹페이지 텍스트 -> Documents
docs = loader.load()

print(len(docs))
print(len(docs[0].page_content))
print(docs[0].page_content[5000:6000])
```

RAG (Retrieval-Augmented Generation)

■ Text Split

- 텍스트 분할(Text Split)
- 자연어 처리(NLP) 기술을 활용하여 큰 문서를 처리가 쉽도록 문단, 문장 또는 구 단위로 나누는 작업
- RecursiveCharacterTextSplitter

```
# Text Split (Documents -> small chunks: Documents)
from langchain_text_splitters import RecursiveCharacterTextSplitter

text_splitter = RecursiveCharacterTextSplitter(chunk_size=1000, chunk_overlap=200)
splits = text_splitter.split_documents(docs)

print(len(splits))
print(splits[10])
print(splits[10].page_content)
print(splits[10].metadata)
```

RAG (Retrieval-Augmented Generation)

■ Text Split

Splitter 클래스	특징
RecursiveCharacterTextSplitter	가장 많이 사용
CharacterTextSplitter	문자 기준
TokenTextSplitter	Token 기준
SentenceTransformersTokenTextSplitter	SentenceTransformer
MarkdownHeaderTextSplitter	Markdown
MarkdownTextSplitter	Markdown
HTMLHeaderTextSplitter	HTML
HTMLSectionSplitter	HTML
PythonCodeTextSplitter	Python 코드
RecursiveJsonSplitter	JSON
LanguageTextSplitter	언어별 코드

RAG (Retrieval-Augmented Generation)

■ Embedding

- 텍스트를 고정된 길이의 숫자 벡터(Vector)로 변환하는 클래스
- 문장의 의미를 벡터 공간에 표현하여 의미적으로 비슷한 문장이 가까운 위치에 있도록 만듭니다.
- RAG(Retrieval-Augmented Generation), 유사도 검색, 문서 검색, 추천 시스템에서 필수

Embedding 클래스	특징
OpenAIEmbeddings	OpenAI API 사용, 높은 정확도, 다국어 지원, RAG에서 가장 많이 사용
OllamaEmbeddings	로컬 환경에서 실행, 무료, 개인정보 보호에 유리, 인터넷 없이 사용 가능
HuggingFaceEmbeddings	Hugging Face 모델 사용, 다양한 모델 선택 가능, 로컬 실행 가능
AzureOpenAIEmbeddings	Azure 환경에서 OpenAI 서비스를 사용할 때 이용,
GoogleGenerativeAIEmbeddings	Gemini 기반 임베딩

HuggingFaceInferenceAPIEmbeddings, VertexAIEmbeddings, BedrockEmbeddings, VoyageAIEmbeddings, CohereEmbeddings, FastEmbedEmbeddings

RAG (Retrieval-Augmented Generation)

■ Indexing

- 검색 시간을 단축시키고, 검색의 정확도를 높이는 데 중요한 역할을 합니다.
- LangChain 라이브러리를 사용하여 텍스트를 임베딩으로 변환하고, 저장 후, 저장된 임베딩을 기반으로 유사성 검색을 수행

```
# OpenAI의 임베딩 모델을 사용하여 텍스트를 벡터로 변환하고, 이를 Chroma 벡터저장소에 저장
# Indexing (Texts -> Embedding -> Store)
from langchain_community.vectorstores import Chroma
from langchain_openai import OpenAIEmbeddings

vectorstore = Chroma.from_documents(documents=splits, embedding=OpenAIEmbeddings())

docs = vectorstore.similarity_search("격하 과정에 대해서 설명해주세요.")
print(len(docs))
print(docs[0].page_content)
```

RAG (Retrieval-Augmented Generation)

■ Retrieval & Generation

- 사용자의 입력을 바탕으로 쿼리를 생성하고, 인덱싱된 데이터에서 가장 관련성 높은 정보를 검색합니다
- LangChain의 retriever()를 사용
- vectorstore.as_retriever() 는 Chroma 벡터 스토어를 검색기로 사용하여 사용자의 질문과 관련된 문서를 검색
- format_docs()는 검색된 문서들을 하나의 문자열로 반환합니다.

```
.....
prompt = ChatPromptTemplate.from_template(template)
# LLM
model = ChatOpenAI(model='gpt-4o-mini', temperature=0)
# Rretriever
retriever = vectorstore.as_retriever()
# Combine Documents
def format_docs(docs):
    return '\n\n'.join(doc.page_content for doc in docs)
# RAG Chain 연결
rag_chain = ( {'context': retriever | format_docs, 'question': RunnablePassthrough() }
             | prompt
             | model
             | StrOutputParser()
             )
.....
```

RAG (Retrieval-Augmented Generation)

■ LangChain.Document Loader

- 사전지식이 필요한 지식 기반의 태스크, 정보 검색, 데이터 처리 작업 등을 처리
- 효율적으로 문서 데이터를 수집하고, 사용 가능한 형식으로 변환
- 웹 페이지, PDF 파일, 데이터베이스 등 다양한 소스에서 문서를 불러올 수 있습니다.
- 불러온 문서 데이터를 분석하고 처리하여, 랭체인이 다른 모듈이나 알고리즘이 처리하기 쉬운 형태로 변환합니다.
- 불필요한 데이터를 제거하거나, 구조를 변경할 수 있습니다.

RAG (Retrieval-Augmented Generation)

■ Document Loader

Document	Document Loader 클래스
PDF	PyPDFLoader, PDFPlumberLoader (표 추출 우수) PyMuPDFLoader(속도 우수) , UnstructuredPDFLoader(복잡한 PDF)
Word	Docx2txtLoader. UnstructuredWordDocumentLoader
Excel	UnstructuredExcelLoader
CSV	CSVLoader
TEXT	TextLoader
Web	WebBaseLoader, RecursiveUrlLoader, SitemapLoader
Markdown	UnstructuredMarkdownLoader
JSON	JSONLoader
XML	UnstructuredXMLLoader
Directory	DirectoryLoader
	GitLoader

RAG (Retrieval-Augmented Generation)

■ LangChain.Document Loader

- **WebBaseLoader** : 특정 웹 페이지의 내용을 로드하고 파싱하기 위해 설계된 클래스
- web_paths 매개변수 : 로드할 웹 페이지의 URL을 단일 문자열 또는 여러 개의 URL을 시퀀스 배열로 설정
- bs_kwargs 매개변수 : BeautifulSoup을 사용하여 HTML을 파싱할 때 사용되는 인자들을 딕셔너리 형태로 제공
- bs4.SoupStrainer를 사용하여 특정 클래스 이름을 가진 HTML 요소만 파싱할 수 있습니다

```
import bs4
from langchain_community.document_loaders import WebBaseLoader
# 여러 개의 url 지정 가능
url1 = "https://blog.langchain.dev/customers-replit/"
url2 = "https://blog.langchain.dev/langgraph-v0-2/"

loader = WebBaseLoader(
    web_paths=(url1, url2),
    bs_kwargs=dict(
        parse_only=bs4.SoupStrainer(
            class_=("article-header", "article-content")
        )
    ),
)
docs = loader.load()
len(docs)
```

RAG (Retrieval-Augmented Generation)

■ LangChain.Document Loader

- TextLoader : 텍스트 파일의 내용을 랭체인 Document 객체로 변환하고 이를 리스트 형태로 반환
파일 경로를 인자로 받아 해당 파일의 내용을 불러온 후, load 메소드를 통해 파일의 내용을 담고 있는 Document 객체로 변환

```
from langchain_community.document_loaders import TextLoader

loader = TextLoader('history.txt')
data = loader.load()

print(type(data))
print(len(data))
print(data)
print(len(data[0].page_content))
print(data.metadata)
```

RAG (Retrieval-Augmented Generation)

■ LangChain.Document Loader

- DirectoryLoader : 디렉토리 내의 모든 문서를 로드
- DirectoryLoader 인스턴스를 생성할 때 문서가 있는 디렉토리의 경로와 해당 문서를 식별할 수 있는 glob 패턴을 지정합니다.

```
!pip install unstructured
```

```
#현재 작업 디렉토리( ./) 내의 모든 .txt 파일을 찾는 작업
from langchain_community.document_loaders import DirectoryLoader

loader = DirectoryLoader(path='./', glob='*.txt', loader_cls=TextLoader)

data = loader.load()
print(len(data))
print(data[0])
```

RAG (Retrieval-Augmented Generation)

■ LangChain.Document Loader

- CSVLoader 클래스 : CSV 파일에서 데이터를 로드합니다.
- CSV 파일의 각 행을 추출하여 서로 다른 Document 객체로 변환합니다
- 문서 객체로 이루어진 리스트 형태로 반환합니다.
- CSV 관련 설정을 csv_args 매개변수를 사용하여 설정

```
from langchain_community.document_loaders.csv_loader import CSVLoader
```

```
loader = CSVLoader(file_path='한국주택금융공사_주택금융관련_지수_20160101.csv', encoding='cp949')  
data = loader.load()  
print(len(data))
```

```
loader = CSVLoader(file_path='한국주택금융공사_주택금융관련_지수_20160101.csv', encoding='cp949' ,  
source_column='연도')  
data = loader.load()  
print(data[0])
```

```
loader = CSVLoader(file_path='한국주택금융공사_주택금융관련_지수_20160101.csv', encoding='cp949' ,  
csv_args={'delimiter': '\n', })  
data = loader.load()  
print(data[0])
```


RAG (Retrieval-Augmented Generation)

■ LangChain.Document Loader

- PyPDFLoader : PDF 파일에서 텍스트를 추출합니다.

```
!pip install -q pypdf
```

```
from langchain_community.document_loaders import PyPDFLoader
```

```
pdf_filepath = '000660_SK_2023.pdf'  
loader = PyPDFLoader(pdf_filepath)  
pages = loader.load()
```

```
print(len(pages))  
print(data[0])
```